

# Scalable Large-Scale Multi-Granularity Code Clone Detection via Clustering Search and Pre-trained Models

Yifan An<sup>†</sup>, Yunlong Ma<sup>†</sup>, Xiang Gao\* and Hailong Sun

**Abstract**—Code cloning is a common phenomenon in software development, which reduces developers’ programming efforts but also poses risks of defect inheritance. Clone detection locates exact or similar pieces of code within or between software systems. With the amount of source code increasing steadily, efficient and large-scale clone detection has become a necessity. Moreover, code clones may occur at various levels of code granularity, e.g., file, function, and block level, which pose more challenges for efficient clone detection. Although numerous methods have been proposed to detect code clones at different granularities, they often suffer from low detection efficiency, false positive results and are typically limited to identifying clones at a specific granularity. In this paper, we introduce an efficient clone detection, named MGCD, to detect code clones among large-scale codebases. Specifically, we embed function-level code into vectors using a pre-trained model and perform clustering search with the IVF\_Flat algorithm to identify clone candidates. These candidates are then filtered through an entropy-based method to enhance accuracy and avoid false positive results. Moreover, we leverage the information from function-level clone detection results to further conduct file and block level clone detection. We evaluate our approach on the BigCloneBench benchmark. Experimental results show that our approach only takes 0.23 ms to search clone results among 800,000 functions and achieves high precision and recall.

**Index Terms**—Code clone detection, multi-granularity, large language model

## I. INTRODUCTION

**I**N software development, to fully leverage existing source code resources, code reuse often occurs through the act of copying and pasting code to another location with little or no modification, i.e., a practice known as code cloning. Studies have shown that 20-50% of large software systems contain cloned code [1], [2], highlighting the prevalence of this issue. In the era of open-source software, the issue of data volume has become increasingly prominent. According to GitHub’s official 2023 Octoverse report [3], the number of repositories on GitHub increased by 27%, exceeding 420 million, and this number is expected to continue growing at an even faster rate. The open-source nature of software not

only increases the volume of code but also contributes to the proliferation of code clones.

Code clones can lead to propagation of errors or vulnerabilities across different software, thereby substantially increasing maintenance costs. For example, a code snippet containing bugs can spread to other programs if copied, thereby transferring those bugs from one program to another and potentially incurring immeasurable costs to fix them.

Moreover, applying Large language Model (LLM) for code generation has gained increasing attentions. LLM has been widely used for code completion, however, they carry the risk of generating code by drawing from existing repositories [4]. This phenomenon of unknowingly cloned code can amplify challenges related to vulnerability and bug inheritance. Unlike traditional post-development clone detection methods, when employing LLM-based coding assistants, it is crucial to provide developers with immediate feedback directly following the generation of code by LLM. The guidance helps ensure that any cloned code are addressed promptly, supporting a more compliant and efficient development process. For such a reason, it is important to detect code clones efficiently in source code [5].

Numerous significant studies have proposed innovative methods for detecting code clones. Most of these methods share a common underlying logic: transforming code into simplified representations, such as tokens, syntax trees, or graphs. Subsequent comparisons are performed using these representations. Depending on the type of representation, these methods are referred to as token-based, tree-based, or graph-based, respectively. Although these methods have shown promising results, they each have their limitations. Firstly, token-based methods [6], [7], [8], [9], [10], [11] are characterized by their low complexity and high scalability. However, they represent source code merely as arrangements of tokens, completely disregarding the syntactic and semantic information embedded within the code. Tree and graph-based methods [12], [13], [14], [15], [16], [17], [18], such as representing code as Abstract Syntax Tree (AST) or Control Flow Graph (CFG), effectively capture the syntactic features of programs and some semantic information. Yet, these approaches still lack a deep understanding of the program’s semantics. Additionally, these methods employ tree-matching algorithms or graph mining techniques for clone detection, which involve considerable computational overheads. As a result, they often struggle to operate effectively within large-scale datasets. With the advancement of deep learning technologies [19], in recent

<sup>†</sup> Co-first authors    \* Corresponding author

Yifan An is with the State Key Laboratory of Complex & Critical Software Environment (SKLCCSE), Beihang University, China (email: anyifan@buaa.edu.cn)

Yunlong Ma, Xiang Gao and Hailong Sun are with the State Key Laboratory of Complex & Critical Software Environment (SKLCCSE), Beihang University, China and Hangzhou Innovation Institute of Beihang University, China (email: yunlong\_ma@buaa.edu.cn, xiang\_gao@buaa.edu.cn, sunhl@buaa.edu.cn)

years there has been the emergence of numerous learning-based techniques [20], [21], [22], [23], [24] for code clone detection. These methods employ deep neural networks to determine the presence of code clones. However, deep learning-based approaches are limited by the size and quality of the training dataset. Moreover, given a piece of code snippet and a codebase, learning-based approaches usually search for clones via pairwise matching. The vast number of comparisons and the subsequent redundant comparisons constitute the primary challenges in achieving efficient clone detection.

In general, the limitations of existing studies can be summarized as follows:

- 1) **Inefficient Detection.** As mentioned earlier, the reasons for inefficient detection include computational overhead of tree and graph algorithms, as well as the slow pairwise comparisons. The overhead of performing pairwise comparisons in large codebases is often unacceptable.
- 2) **False positives.** Useless clones typically follow simple, fixed writing patterns, such as single lines of code or sequences of numbers used for table initialization. For instance, methods that simply return a constant are often identified as useless clones of each other. According to Kamiya et al. [9], these useless clones are equivalent to false positives, which can adversely affect the outcomes of clone detection. Such useless clones appear frequently in programs, often constituting a massive portion of the detected clones, which leads to contamination of the overall results. To address this issue, existing studies use a straightforward method to filter useless clones by predefining a minimum number of code lines as threshold [10], [12], [25], [26], [27]. They only detect code snippets with a number of lines exceeding the threshold, while those with fewer lines are discarded. However, this simplistic approach has notable limitations, as it overlooks the semantics of the code. Hence, it may inadvertently exclude code segments with complex logic.
- 3) **Lack of support for multi-granularity clone detection efficiently.** Current approaches can be categorized by granularity type into fixed granularity methods and free granularity methods. Fixed granularity methods detect clones only at a predetermined level, such as function-level or block-level. In contrast, free granularity methods can identify clones across any code snippet including file, function or block clones. However, both approaches have inherent limitations. Fixed granularity methods may miss clone results at other granularities since they are restricted to detecting clones at a single predefined level. In contrast, free granularity methods often require significantly more detection time, as they need to analyze all files, functions, and blocks comprehensively.

To address these issues, in this paper, we introduce an efficient, multi-granularity clone detection framework, MGCD. The key idea is to significantly reduce the number of actual comparisons during clone detection, reduce the reporting of useless clones, and effectively avoid redundant comparisons in multi-granularity clone detection. Firstly, to reduce the number of comparisons, MGCD clusters code data such that

semantically similar code snippets are grouped within the same cluster and build indices. By doing so, when testing code for clones, it first identifies the relevant cluster, and then only the code snippets within that cluster need to be compared. Secondly, to reduce the reporting of useless clones, we propose using entropy to represent the information content of code snippets. By filtering out code snippets with entropy below a certain threshold, we ensure the provision of meaningful detection results. Thirdly, to avoid redundant comparisons in multi-granularity detection, MGCD first performs clone detection at the function level and then uses the results from function-level detection to support file-level and block-level detection, effectively enhancing the efficiency of the latter two.

The contributions of this paper are as follows:

- We propose a framework, MGCD, to perform scalable and effective clone detection in large-scale codebases under multiple granularity.
- We utilize a novel clone searching approach to minimize the number of searches, along with a filtering approach to exclude useless clones based on entropy.
- We evaluate MGCD using the BCB\_reduced [28] dataset. Experimental results show that MGCD achieves high recall and precision on all types of clones. MGCD only takes 0.23 ms to search clone results among 816,526 functions.
- We make our implementation open-source available at <https://github.com/AnYiFan117/MGCD>.

## II. BACKGROUND

Before presenting the technical details, we first provide some background on IVF\_Flat and entropy.

### A. IVF\_Flat

The IVF\_Flat algorithm [29] is a commonly used indexing technique for large-scale approximate nearest neighbor (ANN) search in high-dimensional vector spaces, such as those derived from images, text, or code. It integrates two components: an Inverted File Index (IVF) and a Flat vector storage. The IVF component partitions the dataset into multiple clusters using a coarse quantizer, typically via k-means, and assigns each vector to its nearest cluster centroid. During query time, only a subset of the most relevant clusters is searched, significantly reducing computational cost. The Flat component stores the original vectors within each cluster without compression or transformation, allowing for precise distance calculations during the fine-grained search phase. By combining efficient cluster-based indexing with exact local search, IVF\_Flat strikes a balance between search speed and accuracy.

### B. Entropy

In information theory, entropy serves as a measure of the uncertainty in information content, providing profound insights across various disciplines to aid in the understanding and management of complex systems. Entropy (denoted as  $H$ ) is calculated based on the probability distribution of the

outcomes in a dataset. Shannon's formula for entropy, often referred to as Shannon entropy [30], is expressed as:

$$H(X) = - \sum_{i=1}^n P(x_i) \log_b P(x_i) \quad (1)$$

where  $P(x_i)$  represents the probability of occurrence of an event  $x_i$  from the set of all possible events  $X$ , and  $n$  is the number of events in the set  $X$ . The base  $b$  of the logarithm depends on the unit of entropy; commonly,  $b = 2$  results in entropy measured in bits. In our approach to code clone detection, we apply the concept of entropy to measure the informational complexity of the source code. This enables the use of a filtering strategy in MGCD.

### III. APPROACH

In this section, we describe the detailed approach for improving clone detection efficiency. We introduce our clustering algorithms and our novel approach for filtering and demonstrate how to detect multi-granularity clones.

Figure 1 illustrates the overall architecture of the proposed technique. First, MGCD embeds the codebase into a vector space, where each vector represents a function-level code snippet. These vectors are indexed into a vector database, which includes not only the embedding vectors but also the clustering and inverted indexing structures for efficient retrieval. The input code to be detected is similarly embedded into a vector, after which MGCD employs IVF\_Flat search to retrieve clone candidates from the database. Second, to filter useless results, we propose using a filtering approach based on entropy. Finally, MGCD performs multi-granularity clone detection. At the file level, it computes similarities between entire files using the results of function-level detection. At the block level, it first segments selected functions based on semantic structure, then performs clone detection over the resulting code blocks using the same function-level detection pipeline.

#### A. Function-level Clone Detection to Obtain Candidates Using IVF\_Flat Search

a) *Utilization of IVF\_Flat*: Efficiently obtaining clone candidates is a crucial task in code clone detection. Many high-recall and high-precision neural network-based Clone Detection Tools (CDTs) use pair-wise comparisons, which result in poor efficiency [20], [24]. Deckard tried to adapt a Locality Sensitive Hashing (LSH) index to cluster similar vectors. Different from existing work, we employ a high-efficiency cluster search to obtain clone candidates.

Clustering refers to the grouping of units with similar characteristics. In the context of code clone detection, distinct code segments that share similar semantics or structural patterns are grouped into clusters. This concept of clustering helps organize and present the results of code clone detection in a standardized way. Additionally, clustering enhances the overall understanding of the clone detection process and offers an effective approach to reducing the overhead involved. Roy et al. [31] have shown that organizing clone pairs into clusters significantly reduces data volume. IVF\_Flat algorithm [29] has

been proven to be an efficient algorithm for clustering search. Hence, we adapt the IVF\_Flat algorithm for efficient clone detection.

We will explain how IVF\_Flat works to generate clusters and do cluster searching below. To show our improvement compared to other famous cluster search such as LSH used in Deckard [12], we also made an ablation experiment in Section IV-B.

As an offline preparation step, MGCD parses the given codebase and translates each code snippet into a representation vector using a code embedding technique. The representation vectors are stored in a vector database, denoted as  $V$ . Subsequently, MGCD employs IVF\_Flat algorithm [29] to complete the search process. First, MGCD partitions vector database  $V$  into a set of clusters (denoted as  $C = \{c_1, c_2, \dots\}$ ) using the K-means algorithm [32].

$$\{c_1, c_2, \dots\} = K\text{-means}(V) \quad (2)$$

Vectors in the same cluster exhibit high similarity to one another, thereby streamlining the search process by reducing the search space to more likely candidates. Each cluster  $c_i$  is composed of a list of vectors. The list of vectors in cluster  $c_i$  is denoted as  $\{c_i^1, c_i^2, \dots\}$ . Each cluster  $c_i$  has a centroid, denoted as  $cc_i$ , which is also calculated using the K-means algorithm [32]. The centroid  $cc_i$  generated by K-means is a vector that represents all the vectors in cluster  $c_i$ . An inverted list  $L$  is created to represent the mapping relationship between  $cc_i$  and  $c_i$ :

$$L = \{(cc_i \mapsto \{c_i^1, c_i^2, \dots, c_i^{n_i}\}) | i = 1, 2, \dots, |C|\} \quad (3)$$

Inverted list  $L$  provides a data structure in which the vector list  $c_i$  can be easily found using the centroid  $cc_i$ .

After the clusters are obtained and the inverted list  $L$  is created, the code clone search is performed in two steps. First, for a code snippet to be detected, MGCD translates it into a representation vector  $x$  by employing the same code embedding approach used in the preprocessing stage. Then, MGCD calculates the L2 distance between  $x$  and centroid of each cluster to select the  $m$  most closest clusters, as illustrated in Equation(4),

$$C_{IVF} = m\text{-argmin}_{cc \in \{cc_i\}_{i=1,2,\dots,|C|}} \|x - cc\|_2 \quad (4)$$

where  $m$  is the number of clusters considered during the search. Small L2 distance between  $x$  and centroid  $cc_i$  indicates that  $x$  is closely aligned with  $cc_i$  in vector space. As illustrated in Equation(3), we can easily find  $c_j$  by  $cc_j$  in  $C_{IVF}$  using inverted list  $L$ . Hence, MGCD performs a flat search [29] through the  $m$  similar clusters whose centroids are in  $C_{IVF}$ . Then, the L2 distance between  $x$  and  $\{c_j^1, c_j^2, \dots\}$  for  $j = 1, 2, \dots, m$  is calculated to select the *top-K* closest vectors, which is then saved as  $L_{IVF\_Flat}$ .

$$L_{IVF\_Flat} = K\text{-argmin}_{c_j^k \in c_j, k=1,2,\dots,n_j, j=1,2,\dots,m} \|x - c_j^k\|_2 \quad (5)$$

Assume  $n$  is the number of vectors within each cluster. By employing IVF\_Flat algorithm, the search complexity within a single cluster is  $O(n)$ . Since the algorithm only probes a constant number of nearest clusters (typically much smaller

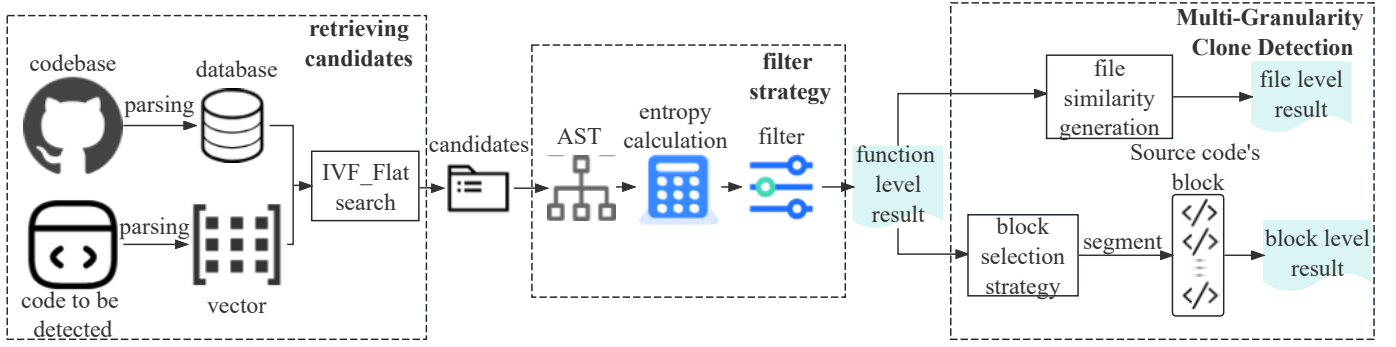


Fig. 1: MGCD's clone detection process

than the total number of clusters), the overall search remains efficient, especially when the database  $V$  is large. While the theoretical complexity is linear with respect to  $n$ , we acknowledge that computing L2 distances between high-dimensional vectors incurs non-trivial constant factors. Nevertheless, this cost is significantly reduced compared to exhaustive search over the entire vector space. The clone candidates retrieved through this step form the input for further processing.

*b) Fine-tune model:* MGCD relies on neural network models to produce embedding. MGCD can directly reuse pre-trained models. To further increase detection accuracy, we can also fine-tune the pre-trained models. Different from existing NN-based clone detection tools [25], which take a pair of codes as input and produce a true/false label, in our context, the model takes only one piece of code and outputs an embedding vector. Hence, we fine-tune the model to perform two separate forward passes, each processing one function independently, rather than a single pass with paired inputs. For code fragments  $f_1$  and  $f_2$ , we embed them separately using the model to generate their representation vectors  $v_1$  and  $v_2$  as two forward passes:

$$\begin{aligned} v_1 &= \text{Model}(f_1) \\ v_2 &= \text{Model}(f_2) \end{aligned} \quad (6)$$

As mentioned earlier, we employ cluster search for clone detection. The ideal performance of the model is achieved when, given a code fragment  $f_1$ , the model generates a vector close to its clone pair  $f_2$ , while remaining distant from non-clone pairs. Hence, the contrastive loss function is well-suited to meet this requirement. The contrastive loss function is defined as follows:

$$\begin{aligned} L(W, (Y, v_1, v_2)) &= \frac{1}{2N} \sum_{n=1}^N Y \cdot \|v_1 - v_2\|^2 \\ &+ (1 - Y) \cdot \max(m - \|v_1 - v_2\|^2, 0) \end{aligned} \quad (7)$$

where  $W$  represents the trainable weights,  $Y \in \{0, 1\}$  is the label of  $v_1, v_2$ , indicating whether they are clones, and  $\|v_1 - v_2\|$  is the euclidean distance between  $v_1$  and  $v_2$ . The parameter  $m$  denotes the margin, which defines the minimum required distance between non-clone pairs. For positive samples ( $Y = 1$ ), the loss function simplifies to  $Y \cdot \|v_1 - v_2\|^2$ , aiming to minimize the distance between

$v_1$  and  $v_2$ , ensuring they are as close as possible when they are clones. For negative samples ( $Y = 0$ ), the loss function becomes  $(1 - Y) \cdot \max(m - \|v_1 - v_2\|^2, 0)$ , which seeks to maximize the distance between non-clone pairs, ensuring that their distance exceeds the margin. Once the distance surpasses the margin, the loss becomes zero.

After computing the loss, we perform the backward pass. During this step, the gradients of the loss with respect to the model's parameters are calculated using the chain rule of differentiation. Finally, these gradients are passed to an optimizer, which updates the model's parameters to improve its performance. Additionally, we utilize a contrastive loss function to fine-tune the model to generate embedding for clone detection purpose.

### B. Post-processing Candidates With Filter

After obtaining the clone candidates, it is crucial to undertake further post-processing in subsequent steps to enhance the accuracy of the results [33]. In this section, we explain our approach to filter clones using an automated heuristic approach.

```

=====
Clone Reported by Nicad
=====
public String ID(){
    return "StdThinGrid";
}

public String ID(){
    return "StdArea";
}

public String displayText(){
    return "";
}

public String toString(){
    return "";
}

```

Fig. 2: Clone results reported by Nicad

*1) Useless clone detection results.:* Kamiya et al. [9] argue that while some clone detection results are indeed clones, they may not be useful to users.

Figure 2 presents a typical example of a "useless" clone, where the four code snippets are identified as clones of each other by the clone detection tool Nicad [34]. However, each snippet contains only a single return statement. Although technically classified as type-2 clones according to the definition, reporting such clone results is often meaningless.

Tools like SourcererCC [10] and CCAAligner [11] allow users to specify a minimum number of lines of code (minLOC) for clone detection, aiming to address certain issues through

this approach. However, using `minLOC` as a filtering criterion is not sufficiently precise [31]. Specifically, functions or code segments shorter than the specified `minLOC` threshold will not be recognized as clones, potentially overlooking important clone instances. For example, setting `minLOC` to 1 can correctly filter out `return 1`; but incorrectly filter `result=sum(i*2 for i in range(10) if i%2==0)` out when the goal is to detect clones of a function that calculates the sum of the squares of even numbers from 0 to 9. Furthermore, the presence of useless clones is not necessarily caused by having too few lines of code. As illustrated in Figure 3, this function initializes a map with a series of string pairs. Such code snippets might form clones with any other segments that construct a specific map in the same manner. These types of code clones are equally meaningless.

```

=====
Map Initialization
=====
public static Map<String, String> loadKey() {
    Map<String, String> keys = new HashMap<String, String>();
    keys.put("user_2000", "87DLJELZWMC1");
    keys.put("user_2001", "R8AMSXSOWR3N");
    .....
    keys.put("user_2043", "DW63JELCY41K");
    keys.put("user_2044", "TSFRLSUCN31J");
    return keys;
}

```

Fig. 3: Sequence of strings for map initialization

Following an analysis of these useless clone results, we believe the fundamental issue is that these code snippets lack sufficient informational content to be useful to users. Therefore, we have proposed solutions specifically targeting this underlying cause.

#### 2) Filtering code snippets based on entropy calculations.:

To address the issues mentioned above, we consider quantifying the information contained within the source code in a measurable manner and applying a threshold for filtering purposes. First, we define a series of basic statements to represent the code to quantify the information content in the source code after thoroughly analyzing the detection result of existing work. The basic types of statements include *return statement*, *assignment*, *method invocation*, *constructor*, *condition*, *loop*, *throw statement* and *try statement*. These basic statements are obtained through the analysis of source code using AST parsing which completes atomic operations and performs specific tasks, such as assigning a value to a parameter.

After representing the source code as a structure composed of basic statements, we employ an enhanced method for calculating entropy, designed to quantify the informational complexity of each statement within the source code. Traditional entropy calculations generally consider only the frequency of statement occurrences, neglecting the intrinsic structural complexity of the statements themselves. To more accurately reflect the informational complexity of source code, we employ a formula based on informational units:

$$H(X) = - \sum_{i=1}^n p(x_i) \log p(x_i) \quad (8)$$

In this formula,  $X$  denotes the code segment under analysis,  $H(X)$  denotes entropy of  $X$ ,  $x_i$  represents the  $i$ -th basic statement within  $X$ ,  $n$  corresponds to the total number of basic statements in  $X$ , and  $p(x_i)$  indicates the normalized information weight of  $x_i$ .  $p(x_i)$  is determined not merely by the frequency of statements, but by the specific informational content of each statement. Specifically, each statement is assigned a number of informational units; for instance, a simple `return 1`; contributes one informational unit, while a more complex statement such as `return a + b + c`; contributes more informational units. In this context, the number of informational units for each specific statement is determined by the count of AST nodes it contains. By calculating the total number of informational units  $U$  across all statements, we normalize the informational contribution of each statement:

$$p(x_i) = \frac{u(x_i)}{U} \quad (9)$$

where  $u(x_i)$  represents the number of informational units for statement  $i$ , and  $U$  is the sum of informational units for all statements.

---

#### Algorithm 1: Filtering Algorithm

---

**Input:** A code snippet:  $c$ ; Entropy threshold:  $\tau$ ; A set of types of basic statements:  $T$

**Output:** Bool

```

1 function filter(c):
2    $S_{basic} \leftarrow \{s \mid s \in \text{ASTNodeSet}(c) \wedge \text{isBasicStmt}(s)\}$ ;
3    $M \leftarrow \{t \mapsto 0 \mid t \in T\}$ ;
4   if  $S_{basic} = \emptyset$  then return true;
5   else
6     foreach  $s \in S_{basic}$  do
7        $t \leftarrow \text{getType}(s)$ ;
8        $M[t] \leftarrow M[t] + \text{CountASTNode}(s)$ ;
9    $H \leftarrow \text{calculateEntropy}(M)$ ;
10  if  $H \geq \tau$  then return false;
11  else return true;

12 function calculateEntropy(M):
13    $TotalCount \leftarrow 0$ ;
14   foreach  $t \in T$  do
15      $TotalCount \leftarrow TotalCount + M[t]$ ;
16    $H \leftarrow 0.0$ ;
17   foreach  $t \in T$  do
18      $Prob \leftarrow M[t]/TotalCount$ ;
19     if  $Prob > 0$  then
20        $H \leftarrow H - Prob \cdot \log(Prob)$ ;
21  return H;

```

---

Centered on the entropy of code snippets, we have designed and implemented a filtering algorithm to optimize detection results. Algorithm 1 illustrates our filtering approach which takes a code snippet as input and determines whether it should be filtered as the output. Upon obtaining the code snippet, the algorithm initially analyzes its abstract syntax tree and, following our previously defined basic statements decomposes the code snippet into a collection of basic statements (line 2). At this stage, if the resultant set is empty, indicating that the function lacks a body, we consider such code clones to be meaningless. Consequently, they are immediately filtered out. If the set is not empty, we construct a map to store the

information density for each type of statement. The process involves iterating over the elements of the statement set. For each basic statement, we analyze the number of AST nodes it contains as its number of informational units. These values are then added to the corresponding type's value in the map (line 8). This map is used for entropy calculation. We sum all values in the map to obtain the total number of informational units for the entire code snippet (line 15). The information frequency for each type of basic statement is then calculated as the quotient of the value for that statement type in the map and the total informational units (line 18). This calculation process aligns with the formula previously mentioned as Formula 9. Finally, we calculate the entropy of the code snippet using Formula 8 and compare it with a predefined threshold. This threshold, which serves as a filtering criterion, can be adjusted as needed to suit different requirements.

### C. Multi-Granularity Clone Detection

In this section, we provide a brief overview of our strategy for detecting clones across multiple granularities. To ensure time efficiency in multi-granularity detection, it is imperative to mitigate duplicate detection across various granularities. To address this issue, we devise a strategy to support multi-granularity clone detection. Following the acquisition of function-level clone results, we implement two levels of detection. The first level broadens the granularity to yield file-level similarity outcomes, while the second level refines the granularity to obtain block-level clone results.

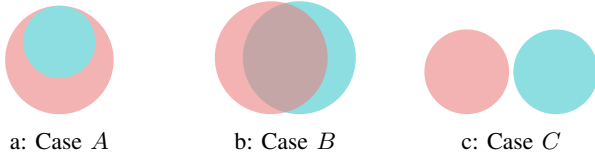


Fig. 4: Three cases of file similarity

1) *File-level clone detection*: The typical clone relationships between two files are illustrated in Figure 4, classified into three inclusion cases. Case A signifies complete inclusion, Case B denotes partial inclusion, and Case C indicates no inclusion.

However, existing methods are unable to report cases of partial inclusion of files. Yuki et al. [35] achieve file-level detection by parsing files for comparison. Their approach can only detect identical copies of files and cannot identify partial inclusions. Moreover, different from text files, changing the order of functions does not affect the semantics of source code files. So directly comparing source files as a whole may produce many false negatives.

To address the issues mentioned above, we have defined a new mode to calculate file-level clone, which is called file similarity. We devised a standardized approach for representing file similarity, along with a conversion technique that translates function-level results into file-level similarity.

MGCD measures file-level similarity by utilizing the function-level clone results. More specifically, the intersection of the two files represents the common LOC, and MGCD uses the proportion of the intersection to represent the similarity.

Let us consider a file denoted as  $x$ , its functions are denoted as  $f_1^x, f_2^x, \dots, f_n^x$ , where  $n$  is the number of functions in  $x$ . Each function  $f_i^x$  has several function-level clone results, noted as  $f'_{i,1}, f'_{i,2}, \dots, f'_{i,m_i}$ , where  $m_i$  is the number of clones to function  $f_i$ . The function-level clone result of  $x$  (denoted as  $R(x)$ ) is the union of each function  $f_i$ 's results, illustrated in Equation (10).

$$R(x) = \bigcup_{i=1}^n \bigcup_{j=1}^{m_i} (f_i^x, f'_j) \quad (10)$$

MGCD then traverses  $R(x)$  to identify the files to which the cloned function  $f'_{i,j}$  belongs. Given a file denoted as  $y$ , MGCD then calculates the similarity between file  $x$  and  $y$  over  $x$  as:

$$\text{sim}(x, y) = \frac{\sum_{f \in \{f_i^x | (f_i^x, f'_j) \in R(x)\}} |f|}{|x|} \quad (11)$$

where  $|x|$  and  $|f|$  refers to the number of lines of code in file  $x$  and function  $f$ , respectively. Similarly, similarity over  $y$  is given by:

$$\text{sim}(y, x) = \frac{\sum_{f \in \{f'_j | (f_i^x, f'_j) \in R(x)\}} |f|}{|y|} \quad (12)$$

where  $|y|$  refers to the number of lines of code in file  $y$ .

When every function in one file has a one-to-one clone relationship with a function in another file—and vice versa—the file similarity is defined as 1, according to our similarity equations. To determine whether two files are considered clones, we introduce a threshold. In our setting, we adopt a strict threshold of 1.0 to maximize precision. That is, we only regard two files as clones if both  $\text{sim}(x, y)$  and  $\text{sim}(y, x)$  are exactly 1.0. This threshold, however, can be adjusted in other applications to accommodate different levels of tolerance for partial similarity.

2) *Block-level clone detection*: In traditional block-level clone detection, it is crucial to extract and examine all code blocks to ensure thorough detection results. However, for large-scale codebases, this approach can lead to data explosion. To address this issue, we propose an approach to obtain block-level clone results. Specifically, we design a strategy to select function pairs that are more likely to exhibit block-level code clones by using the following two criteria: Firstly, the Euclidean distance (L2 distance) between the vectors representing the two functions should not be excessively large, as a substantial distance suggests a low degree of similarity, thereby diminishing the likelihood of shared code segments between them. Secondly, the function pairs should not display high function-level similarity. If two functions are very similar or even fundamentally identical, it is not necessary to perform block-level clone detection.

After filtering, the selected functions are further divided into code blocks for block-level clone detection. Then, how to divide a function into code blocks is the next non-trivial step. Existing approaches divide functions according to sliding window and user-defined  $\text{minLOC}$ . For a function with 50 lines of code, if the  $\text{minLOC}$  is set as 5, there are 45 blocks, such a large number of blocks from a single function

still significantly decreases detection efficiency. To solve this problem, our main insight is that a meaningful clone should exhibit complete functionality. Hence, we define a code block as one that contains complete semantic information. Therefore, we use code segmentation [36] for chunking functions.

To segment functions into multiple meaningful code blocks, in this paper, we propose to large language models (LLMs) to segment functions, which have excellent code understanding ability. LLMs have excellent semantic recognition capabilities. For a given piece of code, they can recognize its semantics and segment the code based on those semantics [36]. After successfully performing semantic segmentation of long functions, MGCD can then use the function-level clone detection to produce block-level clone detection results.

We use DeepSeek [37] in its DeepSeek-V3 version, as it performs the best among all LLMs in code-related tasks [38]. The prompt we use is “*You are a code segmentation program, and you need to complete a code segmentation task. Your input is a long function. You need to split this function into a series of short functions, segmenting them as finely as possible based on their functionality. Your output should be the segmented short functions. Start the task now.*” We set the temperature of the LLM to 0 to ensure deterministic behavior, so that the same input always results in the same code block division. After segmentation, a function with complex semantic information is decomposed into several code blocks, each containing independent and complete semantic information. Finally, we perform clone detection on the segmented code blocks. As with function-level clone detection, we use IVF\_Flat search to detect clones among the code blocks and obtain the final results, which are reported as block-level clone detection results.

#### IV. IMPLEMENTATION AND EVALUATION

In this section, we aim to evaluate MGCD and answer the following research questions:

- **RQ1:** Compared to existing CDTs, what is the performance of MGCD in detecting clones in large-scale codebase?
- **RQ2:** How does our filter approach perform in enhancing precision?
- **RQ3:** How effective is the multi-granularity clone?
- **RQ4:** What is the impact of hyperparameters on the clone detection precision?

##### A. Experimental Settings

1) *Experimental Dataset:* To check the performance of MGCD to detect clones in large-scale database, we choose BigCloneBench [39] as our first dataset since it is the largest and widely used Java clone benchmark [25], [12], [10], [27], [26]. It includes a comprehensive database known as IJA-Dataset [39], which comprises over 25,000 Java open-source projects. For evaluation purposes, we utilize a reduced version of BigCloneBench, known as BCB\_reduced [28] (noted as BCB simply below). This version comprises 13 million lines of code (LOC) and 816,526 functions. Recall is calculated using BigCloneEval (BCE) [28], a tool specifically designed to evaluate clone detection results within the BCB. BCE further

divided Type-3 and Type-4 clone into four subtypes based on line-level and token-level similarity [28]: i) *Very-Strongly Type-3* (VST3) clones have a similarity range of [0.9, 1), ii) *Strongly Type-3* (ST3) clones have a similarity range of [0.7, 0.9), iii) *Moderately Type-3* (MT3) clones have a similarity range of [0.5, 0.7), and iv) *Weakly Type-3/Type-4* (WT3/T4) clones have a similarity range of [0, 0.5). BCE contains over 8 million clone pairs and is configured with the following parameters in the following evaluation: a coverage threshold of 95% for the matcher.

To check the ability of MGCD to detect semantic clones, we choose another widely used dataset named Google Code Jam (GCJ) [24], which is primarily composed of semantic clones. Google Code Jam is derived from an online programming competition held by Google and contains 1669 projects from 12 different competition problems which are written by different programmers. So projects of the same competing problem are almost syntactically different but semantically similar. We use the benchmark presented by [40], with 275,570 semantic clone pairs and 270,000 non-clone pairs.

MGCD implements multi-granularity clone detection at file-level and block-level. To properly evaluate its effectiveness, a dedicated dataset covering both granularities is required. For file-level evaluation, we adopt BCB for clone detection. However, BCB only provides function-level clone annotations, making it impossible to compute recall for file-level detection. For block-level evaluation, there is—to the best of our knowledge—no existing dataset that supports block clone evaluation. To address this gap, we construct a custom dataset by selecting a subset of files from BCB and GCJ dataset. Specifically, we randomly sample 100 files from these sources. To ensure reliable ground truth, two independent annotators manually labeled the block-level clones in the selected files. This dual-annotation approach helps mitigate potential labeling bias and improves the validity of our evaluation.

We would like to clarify that although we primarily evaluated MGCD on BCB and GCJ, it can also be applied to other benchmark datasets such as OJClone [41], SemanticCloneBench [42], and GPT-CloneBench [43]. Since MGCD does not depend on dataset-specific preprocessing, it can be directly applied to these datasets without modification. We did not use SemanticCloneBench for evaluation because MGCD is designed to enhance the efficiency of clone detection, while SemanticCloneBench is designed to evaluate the effectiveness of tools in detecting semantic clones. Because it focuses on a relatively narrow subset of clones, the amount of available data is limited. In total, SemanticCloneBench provides only about 4,000 clone instances for Java, Python, C, and C#, among which just 1,000 are Java clone pairs. This is very small compared to BigCloneBench, which contains 816,526 functions and millions of clone pairs. Consequently, SemanticCloneBench is not sufficiently large or extensible for our use case.

2) *Implementation:* The embedding of source code, the filter, the searching of clone candidates and the obtaining of multi-granularity clone results are implemented in Python. Faiss [44] is used for implementing search indexes, such as IVF\_Flat and LSH. The parameter below is used in MGCD.



- *model* refers to the specific model used for code embedding.
- *nlist* represents the number of clusters generated by IVF\_Flat index. According to the advice of faiss document [45], we set *nlist* to 894 to meet the balance performance.
- *K* represents the number of results returned when searching for clone candidates. In the IVF\_Flat algorithm, the top *K* most similar vectors are returned.
- *nprobe* represents the number of clusters considered during the IVF\_Flat search.
- *distance* refers to threshold. Only if the L2 distance between two vectors is under the *distance* threshold, it will be considered as clone.

In all experiments, MGCD uses the optimal parameters of *model*, *K*, *nprobe* and *distance* as established and validated in RQ4.

Here are the different versions of MGCD evaluated in our experiments:

- MGCD: the standard version, which employs IVF\_Flat for vector search without applying the filter.
- MGCD<sup>w</sup>: a version that disables IVF\_Flat, performing pairwise search instead.
- MGCD<sup>l</sup>: a version that replaces IVF\_Flat with LSH.
- MGCD<sup>ft</sup>: a version that uses a fine-tuned model for vector embedding.
- MGCD<sup>f</sup>: a version that enables the filter to reduce useless clones.
- MGCD<sup>s</sup>: a version that directly detect block-level clones.

MGCD does not activate the filter, as doing so would reduce recall by mistakenly filtering out true positives in the BCB dataset. MGCD<sup>w</sup> and MGCD<sup>l</sup> are analyzed in Section IV-B to assess the effectiveness of IVF\_Flat. MGCD<sup>ft</sup> is included in Section IV-B to ensure a fair comparison with other neural network (noted as NN below) tools. MGCD<sup>f</sup> is used in Section IV-C as an ablation study to examine its effect. MGCD<sup>l</sup> is used in Section IV-D2 as an ablation study to the strategy of block-level clone detection.

3) *Baseline CDTs*: We consider SourcererCC [10], Deckard [12], MSCCD [26], TACC [27], ASTNN [25], RAE [46], CDLH [47], SCDetector [48] and NiCad [34] as our comparison tools.

- **SourcererCC** is a typical token-based CDT that uses an optimized partial index and filtering heuristics to perform large-scale clone detection on a standard workstation.
- **Deckard**, a tree-based CDT, utilizes vectors embedded from tree nodes for comparison.
- **MSCCD**, a grammar-pluggable CDT introduced in recent years, demonstrates the highest language scalability and strong performance in evaluation metrics.
- **TACC** is a tool that combines 12 algorithms to focus on clone detection, excelling in detecting 100MLOC in less time while maintaining strong performance in large-scale clone detection.
- **ASTNN** detects clones relying on an AST-based neural network.
- **RAE**, a recursive autoencoder, is employed to automatically encode ASTs for detecting code clones.

- **CDLH** is a deep learning framework for functional code clone detection using AST-based LSTM and similarity supervision.
- **SCDetector** detects semantic code clones by converting CFGs into tokens and using a Siamese network for similarity learning.
- **NiCad** is a CDT that uses code normalization and text comparison to identify code clones.

We selected SourcererCC and Deckard because they are representative tools for their respective detection approaches and are also widely used for large-scale codebases. MSCCD and TACC were chosen because they have delivered state-of-the-art results in recent years, particularly in handling large-scale codebases. Since MGCD leverages a pre-trained model to embed code, we included some NN tools, such as ASTNN, RAE, CDLH, SCDetector for comparison. Furthermore, to compare the block-level clone detection effectiveness, NiCad is considered since it has the ability to detect block-level clones.

Many recent tools, such like Tritor [40], Toma [49] are not take into account here. Because they are implemented as benchmark-specific prototypes, take JSON-like or id-label inputs instead of source files, and are tightly coupled to particular benchmark pipelines, which makes it difficult to run them on arbitrary projects in a reproducible way.

4) *Metrics to Assess Effectiveness*: Recall and precision are commonly used to assess the effectiveness of CDTs. *F<sub>1</sub>-Score* is used to measure the robustness of MGCD. The calculation of these metrics is performed according to the following equations:

$$recall = \frac{TP}{TP + FN} \quad (13)$$

$$precision = \frac{TP}{TP + FP} \quad (14)$$

$$F_1\text{-Score} = 2 \times \frac{precision \times recall}{precision + recall} \quad (15)$$

TP denotes the number of instances in which the true result was correctly predicted by the tool. FN denotes the number of instances in which the true result was not predicted by the tool. FP denotes the number of instances in which the tool predicted a false result. In our experiment, recall is calculated by BigCloneEval [28]. For precision, since the BCB benchmark only labels partial ground-truth clones, it is impossible to automatically calculate precision. Hence, just as the case with most work in this field [12], [10], [11], [50], we calculate precision by randomly selecting 200 detection results and manually evaluating to determine whether they are real clones.

All the experiments are conducted on a device with 64 cores of 2.3GHz CPU, 128GB RAM, NVIDIA Ampere A100 GPUs, and 40 GB memory. The operating system is Ubuntu 20.04.

*B. RQ1: How Our Approach Compares to Existing Scalable CDTs*

To assess MGCD's effectiveness and efficiency, we measured time efficiency, precision, and recall in detecting clones and compared it with existing tools.



**Experiment design.** To ensure a fair comparison, we divided the effectiveness evaluation into three parts:

- 1) the first part compares MGCD with non-NN tools on BCB.
- 2) the second part compares MGCD with NN tools using a training and evaluation subset derived from BCB.
- 3) the third part compares MGCD with NN tools on GCJ.

This division is necessary because non-NN and NN tools differ fundamentally in both input format and evaluation methodology. Non-NN tools are evaluated directly on the full BCB dataset, while recall is calculated using BCE. In contrast, NN tools require labeled function pairs in the form of "id1, id2, label" for training, which is not compatible with the original structure of BCB. Furthermore, NN tools assess effectiveness by predicting the label of a given function pair and comparing it to ground-truth labels in the evaluation set, making BCE inapplicable for them. To ensure consistency and fairness, we used the same training and evaluation datasets provided by ASTNN [25] for all NN tools, including  $MGCD^{ft}$ . Since the training process is crucial for NN tools, using a unified training set helps control for confounding variables. For  $MGCD^{ft}$ , fine-tuning on this dataset also mitigates the impact of training variability. Additionally, we compare MGCD and  $MGCD^{ft}$  with NN tools on the GCJ dataset. GCJ is specifically designed for NN tools—it includes labeled data—so we perform this comparison using the same training and evaluation split for all tools.

TABLE I: Running time (minutes) comparison of CDTs

| CDT         | Offline Preparing |       | Clone Searching |         |
|-------------|-------------------|-------|-----------------|---------|
|             | Train             | Embed | Search          | Predict |
| SourcererCC | -                 | -     | 105             | -       |
| Deckard     | -                 | -     | 333             | -       |
| MSCCD       | -                 | -     | 52              | -       |
| TACC        | -                 | -     | 29              | -       |
| ASTNN       | 138               | -     | -               | 32      |
| RAE         | 170               | -     | -               | 26      |
| CDLH        | 583               | -     | -               | 45      |
| SCDetector  | 79                | -     | -               | 33      |
| $MGCD^w$    | -                 | 50    | 35              | -       |
| MGCD        | - <sup>1</sup>    | 50    | 6               | -       |
| $MGCD^l$    | -                 | 50    | 14              | -       |
| $MGCD^{ft}$ | 12                | 50    | 6               | -       |

**Efficiency.** Table I shows the time efficiency of clone detection performed by different CDTs on BCB. It is important to note that Table I accounts for the entire runtime of each CDT, even though some fields in the table are left blank. Because different tools follow distinct procedures for detecting clones. For example, SourcererCC does not include a training or embedding step, whereas ASTNN includes a training phase but does not have an embedding phase. As a result, the table only lists the search time for SourcererCC, while for ASTNN, it includes both training and prediction times. As shown in the results, MGCD requires 50 minutes for embedding and 8 seconds for indexing tasks, followed by clone searching time in just three minutes for 816,526 clone detection queries. On

average, it only takes 0.23ms per query. In contrast, Deckard, SourcererCC, RAE, and ASTNN require 333, 105, 157, and 133 minutes, respectively, for inference or detection tasks. The longer times for these tools are due to their reliance on pairwise comparisons, which significantly slows down the process.

Additionally, MSCCD and TACC complete detection in 52 and 29 minutes, respectively, which are faster than MGCD. However, the processes of embedding and indexing are typically one-time efforts, whereas the search process continuously expands with more clone detection queries. MGCD significantly enhances the time efficiency of the search phase, thereby substantially improving the overall time efficiency of code clone detection. This improvement not only boosts efficiency but also enhances the scalability of our approach.

In the ablation experiment on IVF\_Flat search,  $MGCD^w$  required 35 minutes to complete clone detection across all data. This result in a huge increase in time compared to using IVF\_Flat, highlighting the significant impact of IVF\_Flat search efficiency. Besides,  $MGCD^l$  results in a search time of 14 minutes, which is slower than IVF\_Flat. This is reasonable due to the nature of the LSH algorithm. The vectors generated by the pre-trained model are 768-dimensional and densely populated. Since LSH relies on random hash functions to assign vectors to buckets, it requires a large number of hash functions and tables to accurately separate dense vectors, resulting in poor efficiency. Its recall performance is also significantly worse than that of IVF\_Flat. This is because high-dimensional dense vectors exhibit partial similarity that is difficult to capture using hash-based methods, causing many similar vectors to be placed into different buckets. This leads to the observed drop in recall performance.

TABLE II: Recall and precision comparison to other CDT on BigCloneBench (%)

| CDT         | BigCloneBench |     |      |     |     |        | P  | $F_1$ |
|-------------|---------------|-----|------|-----|-----|--------|----|-------|
|             | Recall        |     |      |     |     |        |    |       |
|             | T1            | T2  | VST3 | ST3 | MT3 | WT3/T4 |    |       |
| SourcererCC | 100           | 98  | 93   | 61  | 5   | 0      | 83 | 74.6  |
| MSCCD       | 100           | 98  | 93   | 63  | 7   | 1      | 90 | 77.9  |
| TACC        | 100           | 100 | 98   | 97  | 31  | 2      | 92 | 85.7  |
| Deckard     | 60            | 58  | 62   | 31  | 12  | 0      | 28 | 33.9  |
| MGCD        | 100           | 99  | 95   | 85  | 26  | 2      | 89 | 84.1  |
| $MGCD^w$    | 100           | 99  | 96   | 85  | 27  | 2      | 88 | 82.3  |
| $MGCD^l$    | 100           | 98  | 85   | 41  | 7   | 0      | 84 | 73.5  |
| ASTNN       | 100           | 100 | 96   | 94  | 91  | 88     | 99 | 93.8  |
| RAE         | 100           | 97  | 83   | 72  | 75  | 58     | 76 | 66.6  |
| CDLH        | 100           | 99  | 95   | 92  | 80  | 73     | 91 | 82.0  |
| SCDetector  | 100           | 100 | 95   | 94  | 90  | 85     | 97 | 89.4  |
| $MGCD^{ft}$ | 100           | 100 | 97   | 94  | 90  | 54     | 99 | 71.5  |

**Effectiveness.** Table II reports the clone detection results on BCB, while Table III reports the results on GCJ.

As shown in the first part of Table II, MGCD shows similar recall on T1, T2, and VST3 with SourcererCC, MSCCD, and Deckard, but produces higher recall on type-3 and type-4 clones compared to these tools. Meanwhile, MGCD maintains higher precision. These results highlight MGCD's effectiveness in detecting a wide range of code clones. This is

<sup>1</sup>We do not report the training time here since we use a pre-trained model.

largely due to MGCD's use of a pre-trained model for code embeddings, which helps preserve the semantic information within the code. TACC, on the other hand, employs multiple algorithms that enhance its recall on type-3 clones. Although TACC leads in overall effectiveness, MGCD delivers competitive results, staying close in performance and surpassing TACC in search efficiency, as previously noted. Moreover,  $\text{MGCD}^w$  achieves nearly the same recall as MGCD, indicating that the IVF\_Flat search does not significantly compromise detection accuracy.

The second part in table II presents the comparison results with NN tools. The results show that  $\text{MGCD}^{ft}$  outperforms RAE across all types of clones and performs similarly to ASTNN, SCDetector and CDLH from T1 to MT3, though it experiences a decline in performance for WT3/T4. This difference is expected due to the fundamental distinction between them. In those NN tools' case, the input is a function pair without labels, and the task is simply to determine whether the pair is a clone or not. Since this is a binary classification task, recall can be easily improved by enhancing the model's ability to identify clone pairs. However, MGCD operates differently. It takes only one function as input and identifies clones from the entire dataset, which is essentially a ranking task. The objective is to rank functions based on their similarity. WT3/T4 clones are expected to be ranked lower than type-1, type-2, and type-3 clones. The top  $K$  ranked clone results may not include the labeled ground truth, leading to low recall on WT3/T4. Nonetheless, the precision is 99%, meaning the top  $K$  ranked clones are also very likely to be real clones, which may not be labeled in the dataset.

TABLE III: Recall and precision comparison to other CDT on GoogleCodeJam (%)

| CDT                | GoogleCodeJam |           |       |
|--------------------|---------------|-----------|-------|
|                    | Recall        | Precision | $F_1$ |
| ASTNN              | 87            | 95        | 91    |
| RAE                | 66            | 49        | 56    |
| CDLH               | 70            | 46        | 55    |
| SCDetector         | 87            | 81        | 82    |
| MGCD               | 23            | 94        | 37    |
| $\text{MGCD}^{ft}$ | 99            | 99        | 99    |

Table III shows the result of NN tools and MGCD and  $\text{MGCD}^{ft}$  on GCJ.

As shown in the table, after fine-tuning,  $\text{MGCD}^{ft}$  achieves the best performance among all tools, with both recall and precision reaching 99%. Unfortunately, we cannot report the recall by clone types as in Table II, since the GCJ dataset does not provide annotations for different clone types. Unlike the results in Table II,  $\text{MGCD}^{ft}$  achieves strong performance on GCJ. This is primarily because GCJ is a relatively small dataset compared to BCB, making it easier to retrieve all relevant clone results within a given  $K$  range.

MGCD only achieves 23% recall—the lowest among the compared NN tools—despite maintaining a high precision of 94%. This significant gap is due to the characteristics of the

GCJ dataset, which predominantly contains T3/T4 clones [40]. Such clones often involve structural and semantic changes that are challenging for models not specifically trained on them. This observation aligns with the results on MT3 clones reported in Table II, further indicating that the default MGCD struggles with T3/T4 clones, but can outperform other NN tools after task-specific fine-tuning.

However, it is important to note that the exceptional performance of  $\text{MGCD}^{ft}$  and other NN tools in GCJ and Table II may be partly due to overfitting. Specifically, GCJ and BCB share similar clone distributions, so training or fine-tuning on one may introduce dataset bias, inflating precision and recall. Such results may not generalize well to real-world scenarios. In practical applications, we prioritize minimizing bias and ensuring consistent performance across diverse datasets. Therefore, while  $\text{MGCD}^{ft}$  is included here for fair comparison with other NN tools, we advocate using the default MGCD in real-world use cases due to its better generalization capability.

The experimental results demonstrate that MGCD significantly increases clone detection efficiency, while not sacrificing detection accuracy.

### C. RQ2: Effectiveness of Filtering Approach in Enhancing Precision

In Section III-B, we proposed an entropy-based approach to filter useless clones. We evaluated its effectiveness by constructing a ground truth of useless clones from the BCB, based on expert judgment. This ground truth comprises 88 clones that should be filtered out and 26 that should not, despite being short.

According to Svajlenko et al. [28], the BCB contains 43 subsets of different functionalities. We randomly selected two files from each subset and identified all functions that should and should not be filtered. This resulted in 88 functions that should be filtered and 26 that should not. The 88 instances to be filtered include parameter-setting functions, initialization functions for maps and lists, and return functions. The 26 instances that should not be filtered include recursive functions, functions with complex return expressions, and simple if-statement functions. All these functions are under five lines long and would be mistakenly filtered out under other minLOC settings.

TABLE IV: Performance of filtering useless clone results using different entropy thresholds. filter<sup>u</sup>: rate of filtering useless clone results, filter<sup>r</sup>: rate of filtering real clone results

|                     | Entropy |      |       |      |       |      | line-based |
|---------------------|---------|------|-------|------|-------|------|------------|
|                     | 0       | 0.6  | 0.625 | 0.65 | 0.675 | 0.7  |            |
| filter <sup>u</sup> | -       | 54.5 | 54.5  | 54.5 | 56.8  | 88.6 | 97.7       |
| filter <sup>r</sup> | -       | 7.7  | 7.7   | 15.4 | 15.4  | 84.6 | 100        |

**Results.** Table IV presents the performance of filtering useless clone results using different entropy thresholds and a line-based baseline. We sought to determine whether the filter could effectively exclude useless results while retaining ones

that are real clone results. The results indicate that when the entropy is below 0.6, MGCD can effectively filter out about 54.5% of the useless clone results (shown in row filter<sup>u</sup>), with only 7.7% of the real clone results mistakenly filtered out (shown in row filter<sup>r</sup>). As the threshold increases from 0.6 to 0.7, both filter<sup>u</sup> and filter<sup>r</sup> grow.

This is because, as demonstrated in formula 8 and 9, higher entropy correlates with a larger  $u(x_i)$ , which means there are more information units in the statement. Useless clone results, such as getters and setters, contain fewer information units compared to real clone results, such as binary search algorithm, making them distinguishable by lower entropy. However, some meaningful code snippets, such as initialization routines, wrapper functions, or helper utilities, may have relatively low entropy because they involve repetitive or simple patterns (e.g., setting default values, calling APIs without much computation). This explains the misclassification of useful clones, as these semantically important functions are mistakenly identified as uninformative. Moreover, certain useless clones—such as repetitive boilerplate code performing similar initialization or configuration tasks—may involve complex control flows (e.g., multiple conditional branches, loops, or error handling). These structural complexities increase the entropy of the code snippets, even though their underlying semantics remain simple or repetitive. This accounts for the misclassification of useless clones, as such clones can evade entropy-based filtering despite being of little practical interest.

An important observation is the sharp increase in both filter<sup>u</sup> and filter<sup>r</sup> around the 0.7 threshold. This is because most real clone results fall within the entropy range of 0.65–0.7, and crossing this range begins to eliminate substantial portions of meaningful results. This trade-off highlights that the choice of entropy threshold must be carefully tuned to maintain a high filtering effectiveness without sacrificing too much recall.

**Note on threshold selection:** We begin our table from 0.6 rather than lower values (e.g., 0.1 or 0.2) because thresholds below 0.6 yield either negligible filtering (for very small values) or nearly identical performance to 0.6 (e.g., 0.4–0.5), and thus offer little additional insight. Starting from 0.6 captures the first point at which entropy-based filtering shows a meaningful effect.

To demonstrate the advantage of entropy-based filtering over existing methods, we compare it with line-based filtering, which removes code clones solely based on function length. We choose the line-based threshold to 6 as most of the tools do [34], [10]. As shown in the last column of Table IV, line-based filtering removes 97.7% of useless clone results—but at the cost of filtering out 100% of real clone results, making it an overly aggressive and blunt strategy. This is mainly because the ground-truth clones selected in our dataset are relatively short in length, making them more likely to be misfiltered by line-based filter. In contrast, entropy-based filtering provides a tunable and more selective mechanism, enabling a better balance between filtering false positives and preserving true positives.

We further experimented to determine if the filter improves overall clone detection accuracy. Table V compares the recall and precision of MGCD and MGCD<sup>f</sup> on BCB, using the

TABLE V: Overall recall and precision using filter.

| CDT               | Recall |    |      |     |     |        | Precision | $F_1$ |
|-------------------|--------|----|------|-----|-----|--------|-----------|-------|
|                   | T1     | T2 | VST3 | ST3 | MT3 | WT3/T4 |           |       |
| MGCD              | 100    | 99 | 95   | 85  | 26  | 2      | 89        | 84.1  |
| MGCD <sup>f</sup> | 99     | 99 | 92   | 80  | 23  | 2      | 97        | 84.5  |

best-performing entropy threshold. With the filter, precision improved significantly from 89% to 97%, indicating that many false positives (i.e., non-clone results with low entropy) were successfully excluded. Although recall dropped slightly on several tasks (e.g., VST3: 95% → 92%, ST3: 85% → 80%), we believe this is acceptable given the large gain in precision. The recall decrease is mainly due to BCB itself, since BCB does not distinguish useless clones and marks them as true positives.

These results suggest that entropy is a more reliable signal than code length for filtering low-value clone results, offering a more fine-grained and semantically grounded filtering strategy.

We believe the improvement in precision is due to the presence of many false positive results comprising code with very low information entropy. These are not true clones but appear similar in vector representation after code embedding, leading to misdetection. Moreover, the decrease of recall can be attributed to filtering out functions that should not be filtered, like return statements with expressions, which tend to have low entropy.

The experimental results demonstrate that MGCD's filter approach significantly eliminates useless clones and enhances the precision of results, without significantly sacrificing recall.

#### D. RQ3: Effectiveness of Multi-Granularity Clone Detection

To measure multi-granularity clone detection effectiveness, we conducted experiments on file-level and block-level clone detection. All the datasets used in the experiment are the mentioned ones in Section IV-A1.

1) *Effectiveness of file-level:* MGCD was adapted to perform file-level clone detection on the entire BCB. Unfortunately, the original BCB does not provide ground truth annotations for file-level clones, making it impossible to directly evaluate recall against the ground truth. To facilitate future research, we present our file-level clone detection results on the BCB independently. MGCD detected 97321 file-level clone pairs with 100% precision from the original 20537443 function-level clone results within 18 minutes. Furthermore, we have found 760899 file-level clone pairs which have inclusion cases as shown in Fig 4, which give us more information about the relationship between files in BCB. We have made both the results publicly available in our GitHub repository to support further comparison and research.

2) *Effectiveness of block-level:* We conducted a comparison between Nicad [34], MGCD and MGCD<sup>s</sup> on our dataset described in Section IV-A1. In Section III-C2, we proposed a strategy to select function pairs that are more likely to exhibit

block-level clones. As for implementation, we use a threshold of *distance* to select function pairs and LLM to segment functions.

TABLE VI: Comparison of block-level clone detection between NiCad, MGCD and MGCD<sup>s</sup>

| CDT               | Recall (%) | Precision (%) | Time         |           |
|-------------------|------------|---------------|--------------|-----------|
|                   |            |               | Segmentation | Searching |
| Nicad             | 36.8       | 92            | 0            | 1.2       |
| MGCD              | 94.7       | 94            | 889.5        | 0.2       |
| MGCD <sup>s</sup> | 97.4       | 93            | 2883.6       | 0.5       |

The threshold selection criteria is explained in Section IV-E2. With the threshold set to  $[0.4, 0.6]$ , MGCD achieved a recall of 94.7%, while Nicad achieved a recall of 36.8% with default parameters. MGCD achieved a slightly higher precision (94%) than NiCad (92%), as NiCad often produced false positives by matching structurally similar but semantically unrelated code (e.g., *else* matches *try* blocks). Upon closer inspection, we observe that while NiCad tends to detect true positives primarily involving simple syntactic constructs with minimal semantic content, MGCD is capable of capturing clones through a deeper understanding of functional similarity. In terms of efficiency, MGCD completes the search in just 0.2 seconds, compared to NiCad's 1.2 seconds. However, MGCD additionally requires 889.5 seconds for function segmentation, which NiCad does not perform. The discrepancy stems from fundamental methodological differences: NiCad employs traditional static analysis based on text comparison, while MGCD utilizes a LLM for semantic function segmentation, which is inherently more time-consuming. Despite the longer preprocessing time, MGCD achieves significantly higher recall and precision, yielding results that better capture semantic similarity.

As an ablation study, we also tested a variant: MGCD<sup>s</sup>. While this approach led to a slight improvement in recall, it significantly increased segmentation time and computational overhead. This inefficiency stems from the fact that applying LLM-based segmentation to every file in the dataset is highly time-consuming. In contrast, MGCD achieves comparable recall with substantially lower execution time. Specifically, MGCD effectively reducing the number of files in the dataset by approximately 76% (excludes 73 out of the 96 files). Since the dataset we used contains numerous functions with clone relationships, we believe that the number of functions to be segmented can be further reduced in real-world scenarios.

In addition, to evaluate the stability of the outputs generated by the LLM, we conducted a simple experiment by repeating the same input query three times under identical settings. In particular, we fixed the temperature parameter to 0, which is designed to reduce randomness during text generation as we mentioned in Section III-C2. The results show that the LLM produced identical outputs across all three runs, demonstrating strong stability when the randomness factor is minimized. This observation suggests that, under deterministic settings, the LLM can provide highly consistent results, which is desirable for tasks requiring reproducibility.

The experimental results show that MGCD's multi-granularity approach accurately identifies file-level similarities, provides semantic clone results, and reduces data explosion at the block level without significantly sacrificing recall.

#### E. RQ4: How to Configure Our Approach for Optimal Effectiveness

We measured the impact of different configurations on MGCD's effectiveness.

1) *Impact of model selection*: Since MGCD utilizes pre-trained models to embed function-level code into vectors and determines semantic similarity by calculating the L2 distance between these vectors, the choice of pre-trained model can significantly impact the effectiveness of the tool's clone detection capabilities. Therefore, we selected several pre-trained models, including CodeBERT [51], UniXcoder [52], CodeT5 [53], CodeT5+ [54], that are currently performing well in the field of code representation and conducted clone detection using our framework. We evaluated them on BCB and collected the results.

TABLE VII: Comparison of effectiveness using different pre-trained models.

| Model     | Recall |    |      |     |     |        | Precision | $F_1$ |
|-----------|--------|----|------|-----|-----|--------|-----------|-------|
|           | T1     | T2 | VST3 | ST3 | MT3 | WT3/T4 |           |       |
| CodeBERT  | 100    | 96 | 91   | 78  | 11  | 0      | 85.5      | 77.8  |
| CodeT5    | 100    | 99 | 92   | 77  | 19  | 0      | 87        | 79.9  |
| CodeT5+   | 100    | 99 | 93   | 80  | 21  | 0.3    | 90        | 81.7  |
| UniXcoder | 100    | 99 | 95   | 85  | 26  | 2      | 89        | 84.1  |

**Results.** Table VII presents the clone detection results using various pre-trained models. The results demonstrate that the effectiveness of clone detection indeed varies with different pre-trained models. Among them, UniXcoder, as the most advanced model currently available, achieves the best results. This indicates that if more sophisticated pre-trained models with a deeper understanding of code semantics emerge in the future, the performance of MGCD could be further enhanced.

2) *Impact of distance*: MGCD considers two vectors as code clones if their L2 distance is less than a specific threshold distance. Setting the distance threshold too low may miss many semantic clones, while setting it too high increases the risk of false positives. To determine an optimal distance, we have designed this experiment to analyze the effects of different distance settings.

TABLE VIII: Comparison of effectiveness using different *distance* under the same model UniXcoder

| <i>distance</i> | Recall |    |      |     |     |        | Precision | $F_1$ |
|-----------------|--------|----|------|-----|-----|--------|-----------|-------|
|                 | T1     | T2 | VST3 | ST3 | MT3 | WT3/T4 |           |       |
| 0.3             | 100    | 99 | 93   | 78  | 16  | 0      | 93.5      | 82.1  |
| 0.4             | 100    | 99 | 95   | 85  | 26  | 2      | 89        | 84.1  |
| 0.5             | 100    | 99 | 97   | 89  | 34  | 2      | 68        | 73.4  |
| 0.6             | 100    | 99 | 98   | 93  | 40  | 3      | 52        | 63.7  |

**Results.** Table VIII presents the precision and recall of MGCD with different distance under the same pre-trained

model UniXcoder. The experimental results show that as distance gradually increases, the recall continuously improves while the precision declines. This is because increasing *distance* results in more code pairs with large L2 distances being identified as clones. While this allows the retrieval of additional semantic clones that exhibit larger L2 distances, it also substantially increase the number of false positives, thereby reducing precision. As a trade-off, a threshold of 0.4 achieves a reasonable balance between recall and precision, offering more favorable performance compared to other distance settings.

3) *Impact of  $K$  and  $nprobe$* : MGCD employs the IVF\_Flat search algorithm for clone detection, which has two important parameters,  $K$  and  $nprobe$  [45].  $K$  controls the number of candidate matches returned by search, while  $nprobe$  specifies the number of clusters to examine for each query. Their configuration presents a fundamental trade-off between search efficiency and effectiveness.

TABLE IX: Comparison of effectiveness using different  $nprobe$  in IVF\_Flat search with *distance* set to 0.4,  $K$  set to 500 and UniXcoder

| $nprobe$ | Recall |    |      |     |     |        | P  | $F_1$ | Time  |
|----------|--------|----|------|-----|-----|--------|----|-------|-------|
|          | T1     | T2 | VST3 | ST3 | MT3 | WT3/T4 |    |       |       |
| 1        | 100    | 98 | 82   | 52  | 20  | 1      | 96 | 81.1  | 4.03  |
| 2        | 100    | 99 | 94   | 83  | 25  | 1      | 91 | 83.1  | 4.34  |
| 3        | 100    | 99 | 95   | 85  | 26  | 2      | 89 | 84.1  | 5.99  |
| 4        | 100    | 99 | 95   | 85  | 26  | 2      | 88 | 82.1  | 7.31  |
| 5        | 100    | 99 | 95   | 85  | 27  | 2      | 89 | 82.3  | 8.53  |
| 6        | 100    | 99 | 95   | 85  | 27  | 2      | 86 | 81.4  | 9.37  |
| 7        | 100    | 99 | 95   | 85  | 27  | 2      | 88 | 82.3  | 10.86 |
| 8        | 100    | 99 | 95   | 85  | 27  | 2      | 88 | 82.3  | 11.51 |
| 9        | 100    | 99 | 95   | 85  | 27  | 2      | 85 | 81.0  | 12.83 |

**Results.** Table IX presents the results for several practical values of  $nprobe$ . As  $nprobe$  increases, recall improves significantly in the early stages, but eventually stabilizes at a certain level. Precision keeps high at start, but tends to decrease as  $nprobe$  increases and eventually stabilizes around 88-91%. This indicates that while more results are retrieved as  $nprobe$  grows, some additional results might not be as relevant, leading to a slight drop in precision. The search time increases as  $nprobe$  grows, highlighting the trade-off between recall, precision, and search efficiency. Considering this trade-off, setting  $nprobe$  to 3 offers a good balance between efficiency and recall, making it a practical choice.

TABLE X: Comparison of effectiveness using different  $K$  in IVF\_Flat search with *distance* set to 0.4,  $nprobe$  set to 3 and UniXcoder

| $K$  | Recall |    |      |     |     |        | P   | $F_1$ | Time |
|------|--------|----|------|-----|-----|--------|-----|-------|------|
|      | T1     | T2 | VST3 | ST3 | MT3 | WT3/T4 |     |       |      |
| 5    | 22     | 5  | 24   | 13  | 5   | 0      | 100 | 25.9  | 3.33 |
| 25   | 63     | 9  | 71   | 45  | 12  | 0      | 95  | 59.0  | 3.76 |
| 100  | 99     | 95 | 88   | 68  | 17  | 1      | 91  | 79.8  | 4.49 |
| 250  | 100    | 98 | 94   | 81  | 23  | 1      | 92  | 82.9  | 5.58 |
| 500  | 100    | 99 | 95   | 85  | 26  | 2      | 89  | 84.1  | 5.99 |
| 1000 | 100    | 99 | 95   | 85  | 26  | 2      | 88  | 82.1  | 6.16 |
| 2500 | 100    | 99 | 95   | 85  | 26  | 2      | 89  | 84.1  | 6.27 |

Table X presents the results for several practical values of  $K$ . As  $K$  increases, recall improves and eventually stabilizes at a certain level, while precision tends to decrease and then stabilize around 90%. Interestingly, the search time does not increase significantly when  $K$  grows from 500 to 2500, in contrast to the more noticeable increase in search time observed in Table IX as  $nprobe$  increases. This is mainly because most of the relevant results, those with a *distance* below the threshold, have already been retrieved when  $K$  is set to 500. As  $K$  increases further, only a small number of additional results are retrieved. Since these results represent a very small portion of the total, the overall increase in search time remains minimal, and both recall and precision stabilize at certain levels. Therefore, setting  $K = 500$  offers a good trade-off between recall, precision, and search efficiency, making it a practical choice for MGCD.

The experimental results demonstrate that MGCD obtains optimal effectiveness when *distance* = 0.4,  $nprobe$  = 3,  $K$  = 500 and UniXcoder is selected as the pre-trained model.

## F. Discussion

In this section, we discuss the limitation of MGCD and the threats that may affect the validity of our evaluation.

*Limitations.* Firstly, our clone detection heavily relies on the effectiveness of the code embedding models. The vectors generated by current pre-trained models are not always stable, which can lead to anomalies where two functions that are very similar may exhibit a large L2 distance. In addition, these models impose a maximum token limit, and for functions with a large number of tokens, the embedding effectiveness is significantly compromised. Secondly, during the filtering process, we calculate the entropy of code snippets to assess their informational content and determine the usefulness of the detected clones. However, since there is still no strict definition of what constitutes a meaningful clone, our filtering method may inadvertently eliminate important clones, which could potentially reduce the recall rate. Thirdly, while we have compared MGCD with both classical and relatively recent NN-based approaches, we did not incorporate the very latest NN-based tools into our experiments. Many of these tools require labor-intensive preprocessing or rely on datasets tailored to their own architectures, which makes a fair comparison under our unified setting impractical. Nevertheless, exploring the integration of the newest NN-based tools remains an interesting direction for future work. Moreover, similar to our experiments on GCJ, fine-tuning the embedding model on a given dataset is expected to further enhance performance. This flexibility demonstrates the robustness of MGCD and suggests its potential to adapt effectively across diverse benchmark datasets.

*Threats to Validity.* MGCD outperforms existing approaches on the BCB\_reduced dataset when detecting function-level clones. However, the BCB\_reduced dataset lacks standardization for granularity, such as file-level or block-level clones. In fact, it is unfortunate to note that there are no publicly

available datasets for clone detection at granularities other than the function level. This fact makes it challenging to evaluate MGCD's performance at file-level and block-level granularities comprehensively. Because the dataset used for evaluation was manually selected, and the verification of the clone detection results for file-level and block-level detections was also performed manually, both aspects may introduce bias. To mitigate this issue, two authors independently reviewed the results and discussed any discrepancies until they reached a consensus.

## V. RELATED WORK

In this section, we will introduce the related work on clone detection, including candidate acquisition strategy, filter strategy, detection granularity and cross-language clone detection involved in the process.

**Code clone detection.** Code clone detection identifies copy-paste code fragments by assessing their similarity. Earlier work transformed code into a unified structural metric to compare similarity and identify clones [9], [12], [55], [56], [57], [58], [59], but this approach struggled with complex and semantic clones. Existing methods also use neural networks for clone detection [20], [24], [60], [22], [46], which perform well with complex and semantic clones but are time-consuming. Recent neural network-based methods have achieved state-of-the-art performance. For example, Toma [49] reports approximately 90% precision and recall, while Tritor [40] and CC2Vec [61] report nearly 100% precision and recall on the BCB dataset. However, these tools are implemented as benchmark-specific prototypes, which makes it difficult to run them on arbitrary projects in a reproducible way. We hope future clone detection tools will expose more generic, file-based interfaces and reproducible pipelines, so that they can be fairly compared and reused on arbitrary codebases.

**Candidate acquisition strategy.** Candidates are clone detection results that have not undergone post-processing. There are various methods to obtain candidates. Tools like CCFinder [9], SourcererCC [10], and CCAAligner [11] tokenize the code and perform maximum subsequence searches to obtain candidates. Jiang et al. [12] generate the AST of the code to identify similar subtrees as candidates, while Zou et al. [18] create the PDG of the code and use graph search methods to obtain candidates. In contrast, we use pre-trained models to vectorize the code and employ clustering search to obtain candidates.

**Filter strategy.** Post-processing of candidates using a filter is an important step in code clone detection. Kamiya et al. [9] state that filters should filter out clone results that are useless to the researcher. Most of the existing work uses `minLOC` for filtering [10], [34], [11]. Chilowicz et al. [62] used a function of call graph for false positive results to improve the accuracy. Wang et al. [63] used a way of comparing the PDGs of the clone results. Kodhai et al. [64] used a text-based comparison to filter the results. Unlike them, we tried a novel approach to filter the code by calculating the information entropy of the code.

**Detection granularity.** Existing clone detection tools generally focus on either fixed granularity [65], [10], [11], [50], [24],

which operates at a predetermined level of granularity, or free granularity [9], [12], which has no fixed granularity. However, both approaches lack scalability and often struggle to detect clones across multiple granularities. Multi-granularity tools report code clones at various levels of granularity. Currently, there are relatively few tools available for multi-granularity clone detection. Yuki et al. [35] proposed a technique that detects clones at multiple granularities, ranging from file-level to block-level clones, using hash values. Yu et al. [66] developed an approach to detect multi-granularity clones in Java code by compiling Java source code into bytecode, which allows for the detection of function-level and block-level clones. We propose methods to detect multi-granularity clones at the file, function, and block levels, providing comprehensive results for researchers to conduct further analysis.

**Cross-language clone detection.** Existing approaches are mainly used to detect clones written in the same programming language [12], [10], [26], [27], [11], [50], [40], [61]. However, it is common to develop programs with the same functionality in different programming languages to support various platforms. Cheng et al. [67] mine cross-language clones from revision histories. Vislavski et al. [68] rely on the SSQSA platform's high-level representation of code, from which it is possible to extract syntactic and semantic characteristics of code fragments, thus enabling full cross-language clone detection. Nafi et al. [69] use features to represent source code and utilize a neural network to detect cross-language clones. Tao et al. [70] use CodeBERT to embed source code and fine-tune the models through contrastive learning.

## VI. CONCLUSION

In this paper, we propose MGCD, a novel code clone detector capable of identifying multi-granularity code clones in large-scale codebases. MGCD utilizes IVF\_Flat search to detect clones among vectors generated by pre-trained models and employs a novel filter approach to refine the results. MGCD also enables multi-granularity clone detection at the file, function, and block levels. We evaluated MGCD using the BCB\_reduced benchmark, and compared it with several scalable, large-scale code clone detectors. The results demonstrate that MGCD outperforms existing approaches in terms of recall and precision, while also maintaining good efficiency. Furthermore, MGCD's filter approach excels at eliminating useless clones and enhancing precision. MGCD's multi-granularity detection capabilities perform well compared to other detectors, reporting comprehensive clone results effectively. For future research, the potential of MGCD to detect other programming languages even cross-language code clones should be validated through additional experiments, and a more comprehensive dataset is required for multi-granularity verification. Our implemented tool is publicly available at <https://github.com/AnYiFan117/MGCD>

## ACKNOWLEDGMENT

This work was supported by the National Key R&D Program of China No 2024YFB4506200 and National Natural Science Foundation of China under Grant No 62202026.

## REFERENCES

- [1] B. S. Baker, "On finding duplication and near-duplication in large software systems," in *Proceedings of 2nd Working Conference on Reverse Engineering*. IEEE, 1995, pp. 86–95.
- [2] S. Ducasse, M. Rieger, and S. Demeyer, "A language independent approach for detecting duplicated code," in *Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99): Software Maintenance for Business Change' (Cat. No. 99CB36360)*. IEEE, 1999, pp. 109–118.
- [3] "GitHub Octoverse 2023," 2024, accessed: Jun. 2024. [Online]. Available: <https://octoverse.github.com/>
- [4] "GitHub, Microsoft, OpenAI fail to wriggle out of Copilot copyright lawsuit," 2024, accessed: date-of-access. [Online]. Available: [https://www.theregister.com/2023/05/12/github\\_microsoft\\_openai\\_copilot](https://www.theregister.com/2023/05/12/github_microsoft_openai_copilot)
- [5] M. Zakeri-Nasrabadi, S. Parsa, M. Ramezani, C. Roy, and M. Ekhtiarzadeh, "A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges," *Journal of Systems and Software*, vol. 204, p. 111796, 2023.
- [6] N. Göde and R. Koschke, "Incremental clone detection," in *2009 13th European conference on software maintenance and reengineering*. IEEE, 2009, pp. 219–228.
- [7] Y. Golubev, V. Poletansky, N. Povarov, and T. Bryksin, "Multi-threshold token-based code clone detection," in *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2021, pp. 496–500.
- [8] Y.-L. Hung and S. Takada, "CPPCD: A token-based approach to detecting potential clones," in *2020 IEEE 14th international workshop on software clones (IWSC)*. IEEE, 2020, pp. 26–32.
- [9] T. Kamiya, S. Kusumoto, and K. Inoue, "CCFinder: A multilingual token-based code clone detection system for large scale source code," *IEEE transactions on software engineering*, vol. 28, no. 7, pp. 654–670, 2002.
- [10] H. Sajnani, V. Saini, J. Svajlenko, C. K. Roy, and C. V. Lopes, "Sourcecerc: Scaling code clone detection to big-code," in *Proceedings of the 38th international conference on software engineering*, 2016, pp. 1157–1168.
- [11] P. Wang, J. Svajlenko, Y. Wu, Y. Xu, and C. K. Roy, "CCAligner: a token based large-gap clone detector," in *Proceedings of the 40th International Conference on Software Engineering*, 2018, pp. 1066–1077.
- [12] L. Jiang, G. Misherghe, Z. Su, and S. Glondou, "Deckard: Scalable and accurate tree-based detection of code clones," in *29th International Conference on Software Engineering (ICSE'07)*. IEEE, 2007, pp. 96–105.
- [13] Y.-B. Jo, J. Lee, and C.-J. Yoo, "Two-Pass technique for clone detection and type classification using tree-based convolution neural network," *Applied Sciences*, vol. 11, no. 14, p. 6613, 2021.
- [14] H. Liang and L. Ai, "AST-path based compare-aggregate network for code clone detection," in *2021 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2021, pp. 1–8.
- [15] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, "A novel neural source code representation based on abstract syntax tree," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 783–794.
- [16] J. Krinke, "Identifying similar code with program dependence graphs," in *Proceedings eighth working conference on reverse engineering*. IEEE, 2001, pp. 301–309.
- [17] M. Wang, P. Wang, and Y. Xu, "CCSharp: An efficient three-phase code clone detector using modified PDGs," in *2017 24th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2017, pp. 100–109.
- [18] Y. Zou, B. Ban, Y. Xue, and Y. Xu, "CCGraph: a PDG-based code clone detector with approximate graph matching," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 2020, pp. 931–942.
- [19] X. Chen, X. Hu, Y. Huang, H. Jiang, W. Ji, Y. Jiang, Y. Jiang, B. Liu, H. Liu, X. Li *et al.*, "Deep learning-based software engineering: progress, challenges, and opportunities," *Science China Information Sciences*, vol. 68, no. 1, p. 111102, 2025.
- [20] L. Li, H. Feng, W. Zhuang, N. Meng, and B. Ryder, "Cclearner: A deep learning-based clone detection approach," in *2017 IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 2017, pp. 249–260.
- [21] W. Hua, Y. Sui, Y. Wan, G. Liu, and G. Xu, "Fcca: Hybrid code representation for functional clone detection using attention networks," *IEEE Transactions on Reliability*, vol. 70, no. 1, pp. 304–318, 2020.
- [22] V. Saini, F. Farmahinifarahani, Y. Lu, P. Baldi, and C. V. Lopes, "Oreo: Detection of clones in the twilight zone," in *Proceedings of the 2018 26th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*, 2018, pp. 354–365.
- [23] H. Wei and M. Li, "Supervised deep features for software functional clone detection by exploiting lexical and syntactical information in source code," in *IJCAI*, 2017, pp. 3034–3040.
- [24] G. Zhao and J. Huang, "Deepsim: deep learning code functional similarity," in *Proceedings of the 2018 26th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 2018, pp. 141–151.
- [25] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, "A novel neural source code representation based on abstract syntax tree," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 783–794.
- [26] W. Zhu, N. Yoshida, T. Kamiya, E. Choi, and H. Takada, "MSCCD: grammar pluggable clone detection based on ANTLR parser generation," in *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*, 2022, pp. 460–470.
- [27] Y. Wang, Y. Ye, Y. Wu, W. Zhang, Y. Xue, and Y. Liu, "Comparison and evaluation of clone detection techniques with different code representations," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 332–344.
- [28] J. Svajlenko and C. K. Roy, "Bigcloneeval: A clone detection tool evaluation framework with bigclonebench," in *2016 IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 2016, pp. 596–600.
- [29] J. Do, V. C. Ferreira, H. Bobarshad, M. Torabzadehkashi, S. Rezaei, A. Heydarigorji, D. Souza, B. F. Goldstein, L. Santiago, M. S. Kim *et al.*, "Cost-effective, energy-efficient, and scalable storage computing for large-scale AI applications," *ACM Transactions on Storage (TOS)*, vol. 16, no. 4, pp. 1–37, 2020.
- [30] C. E. Shannon, "A mathematical theory of communication," *ACM SIGMOBILE mobile computing and communications review*, vol. 5, no. 1, pp. 3–55, 2001.
- [31] C. K. Roy and J. R. Cordy, "A survey on software clone detection research," *Queen's School of computing TR*, vol. 541, no. 115, pp. 64–68, 2007.
- [32] M. Ahmed, R. Seraj, and S. M. S. Islam, "The k-means algorithm: A comprehensive survey and performance evaluation," *Electronics*, vol. 9, no. 8, p. 1295, 2020.
- [33] C. K. Roy, J. R. Cordy, and R. Koschke, "Comparison and evaluation of code clone detection techniques and tools: A qualitative approach," *Science of computer programming*, vol. 74, no. 7, pp. 470–495, 2009.
- [34] C. K. Roy and J. R. Cordy, "NICAD: Accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization," in *2008 16th IEEE international conference on program comprehension*. IEEE, 2008, pp. 172–181.
- [35] Y. Yuki, Y. Higo, and S. Kusumoto, "A technique to detect multi-grained code clones," in *2017 IEEE 11th International Workshop on Software Clones (IWSC)*. IEEE, 2017, pp. 1–7.
- [36] E. A. AlOmar, M. W. Mkaouer, and A. Ouni, "Behind the intent of extract method refactoring: A systematic literature review," *IEEE Transactions on Software Engineering*, 2024.
- [37] X. Bi, D. Chen, G. Chen, S. Chen, D. Dai, C. Deng, H. Ding, K. Dong, Q. Du, Z. Fu *et al.*, "Deepseek llm: Scaling open-source language models with longtermism," *arXiv preprint arXiv:2401.02954*, 2024.
- [38] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.
- [39] J. Svajlenko, J. F. Islam, I. Keivanloo, C. K. Roy, and M. M. Mia, "Towards a big data curated benchmark of inter-project code clones," in *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 2014, pp. 476–480.
- [40] D. Zou, S. Feng, Y. Wu, W. Suo, and H. Jin, "Tritor: Detecting Semantic Code Clones by Building Social Network-Based Triads Model," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 771–783.
- [41] L. Mou, G. Li, L. Zhang, T. Wang, and Z. Jin, "Convolutional neural networks over tree structures for programming language processing," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 30, no. 1, 2016, pp. 1287–1293.
- [42] F. Al-Omari, C. K. Roy, and T. Chen, "Semanticclonebench: A semantic code clone benchmark using crowd-source knowledge," in *2020 IEEE*



- 14th International Workshop on Software Clones (IWSC). IEEE, 2020, pp. 57–63.
- [43] A. I. Alam, P. R. Roy, F. Al-Omari, C. K. Roy, B. Roy, and K. A. Schneider, “Gptclonebench: A comprehensive benchmark of semantic clones and cross-language clones using gpt-3 model and semanticclonebench,” in *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2023, pp. 1–13.
  - [44] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, “The faiss library,” *arXiv preprint arXiv:2401.08281*, 2024.
  - [45] Facebook AI Research. (2023) Guidelines to choose an index. [Online]. Available: <https://github.com/facebookresearch/faiss/wiki/Guidelines-to-choose-an-index>
  - [46] M. White, M. Tufano, C. Vendome, and D. Poshyanyk, “Deep learning code fragments for code clone detection,” in *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*, 2016, pp. 87–98.
  - [47] H. Wei and M. Li, “CDLH: Supervised deep features for software functional clone detection by exploiting lexical and syntactical information in source code,” in *IJCAI*, 2017, pp. 3034–3040.
  - [48] Y. Wu, D. Zou, S. Dou, S. Yang, W. Yang, F. Cheng, H. Liang, and H. Jin, “SCDetector: Software functional clone detection based on semantic tokens analysis,” in *Proceedings of the 35th IEEE/ACM international conference on automated software engineering*, 2020, pp. 821–833.
  - [49] S. Feng, W. Suo, Y. Wu, D. Zou, Y. Liu, and H. Jin, “Machine learning is all you need: A simple token-based approach for effective code clone detection,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
  - [50] M. Chochlov, G. A. Ahmed, J. V. Patten, G. Lu, W. Hou, D. Gregg, and J. Buckley, “Using a nearest-neighbour, BERT-based approach for scalable clone detection,” in *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2022, pp. 582–591.
  - [51] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, “Codebert: A pre-trained model for programming and natural languages,” *arXiv preprint arXiv:2002.08155*, 2020.
  - [52] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, “Unixcoder: Unified cross-modal pre-training for code representation,” *arXiv preprint arXiv:2203.03850*, 2022.
  - [53] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, “Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” *arXiv preprint arXiv:2109.00859*, 2021.
  - [54] Y. Wang, H. Le, A. D. Gotmare, N. D. Bui, J. Li, and S. C. Hoi, “Codet5+: Open code large language models for code understanding and generation,” *arXiv preprint arXiv:2305.07922*, 2023.
  - [55] K. W. Nafi, B. Roy, C. K. Roy, and K. A. Schneider, “A universal cross language software similarity detector for open source software categorization,” *Journal of Systems and Software*, vol. 162, p. 110491, 2020.
  - [56] M. Sudhamani and L. Rangarajan, “Code clone detection based on order and content of control statements,” in *2016 2nd International Conference on Contemporary Computing and Informatics (IC3I)*. IEEE, 2016, pp. 59–64.
  - [57] K. Uemura, A. Mori, K. Fujiwara, E. Choi, and H. Iida, “Detecting and analyzing code clones in HDL,” in *2017 IEEE 11th International Workshop on Software Clones (IWSC)*. IEEE, 2017, pp. 1–7.
  - [58] A. Sheneamer and J. Kalita, “Code clone detection using coarse and fine-grained hybrid approaches,” in *2015 IEEE seventh international conference on intelligent computing and information systems (ICICIS)*. IEEE, 2015, pp. 472–480.
  - [59] M. Singh and V. Sharma, “Detection of file level clone for high level cloning,” *Procedia Computer Science*, vol. 57, pp. 915–922, 2015.
  - [60] S. Feng, W. Suo, Y. Wu, D. Zou, Y. Liu, and H. Jin, “Machine Learning is All You Need: A Simple Token-based Approach for Effective Code Clone Detection,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
  - [61] S. Dou, Y. Wu, H. Jia, Y. Zhou, Y. Liu, and Y. Liu, “Cc2vec: Combining typed tokens with contrastive learning for effective code clone detection,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1564–1584, 2024.
  - [62] M. Chilowicz, É. Duris, and G. Roussel, “Viewing functions as token sequences to highlight similarities in source code,” *Science of Computer Programming*, vol. 78, no. 10, pp. 1871–1891, 2013.
  - [63] T. Wang, K. Wang, X. Su, and P. Ma, “Detection of semantically similar code,” *Frontiers of Computer Science*, vol. 8, pp. 996–1011, 2014.
  - [64] E. Kodhai and S. Kanmani, “Method-level code clone detection through LWH (Light Weight Hybrid) approach,” *Journal of Software Engineering Research and Development*, vol. 2, pp. 1–29, 2014.
  - [65] U. Manber *et al.*, “Finding similar files in a large file system,” in *Usenix winter*, vol. 94, 1994, pp. 1–10.
  - [66] D. Yu, J. Wang, Q. Wu, J. Yang, J. Wang, W. Yang, and W. Yan, “Detecting java code clones with multi-granularities based on bytecode,” in *2017 IEEE 41st Annual Computer Software and Applications Conference (COMPSAC)*, vol. 1. IEEE, 2017, pp. 317–326.
  - [67] X. Cheng, Z. Peng, L. Jiang, H. Zhong, H. Yu, and J. Zhao, “Mining revision histories to detect cross-language clones without intermediates,” in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, 2016, pp. 696–701.
  - [68] T. Vislavski, G. Rakić, N. Cardozo, and Z. Budimac, “Licca: A tool for cross-language clone detection,” in *2018 IEEE 25th international conference on software analysis, evolution and reengineering (SANER)*. IEEE, 2018, pp. 512–516.
  - [69] K. W. Nafi, T. S. Kar, B. Roy, C. K. Roy, and K. A. Schneider, “Clcdsa: cross language code clone detection using syntactical features and api documentation,” in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 1026–1037.
  - [70] C. Tao, Q. Zhan, X. Hu, and X. Xia, “C4: Contrastive cross-language code clone detection,” in *Proceedings of the 30th IEEE/ACM international conference on program comprehension*, 2022, pp. 413–424.